Tiered Memory Management Beyond Hotness

CS-471: Project Status Report

Mayeul Besnard
EPFL
mayeul.besnard@epfl.ch

Sergio Boffi
EPFL
sergio.boffi@epfl.ch

November 12, 2026

Contents

1 Research Problem **2**

2 Questions, Hypotheses, and Tests **2**
2.1 Methodology and Testing Environment Updates 2
2.2 Current Status of Research Questions 3
2.2.1 Q1: Can hotness be a faithful metric to measure performance? 3
2.2.2 Q2: If not, how can a better metric be defined? 4
2.2.3 Q3: With this metric, can we improve first data placement (SOAR)? . . . 4
2.2.4 Q4: Can these elements improve PolyRack? 4
2.3 Anomalies and Updates to the Hypothesis Tree 4

3 Progress, Results, and Analysis **4**
3.1 Experimental Setup 4
3.2 E1: Hotness versus Criticality 5
3.3 E2: Slowdown Predictors (Preliminary) 6
3.4 SOAR: Feasibility 7

4 Challenges **7**
4.1 Hardware Differences 7
4.2 Tiered Memory Emulation and Baselines 7
4.3 Authors' Artifact 7
4.4 Help Needed from the Teaching Team 7

5 Plan **7**

1 Research Problem

Tiered memory systems combine a small, fast tier (local DRAM) with a larger, cheaper but slower tier (e.g., CXL-attached memory), so the way data is distributed across the two tiers determines how many memory stalls an application suffers. State-of-the-art tiering systems (TPP, NBT, Nomad and Colloid) make this decision based on access frequency, or *hotness*.

TPP (Transparent Page Placement) was designed by Maruf et al. (ASPLOS 2023), and simply uses a lightweight mechanism to promote pages from CXL memory to local DRAM and demote them back.

Nomad (Non-Exclusive Memory Tiering via Transactional Page Migration), proposed by Xiang et al. (OSDI 2024), uses non-exclusive memory placement. Page are not exclusively placed on fast tier or CXL memory, but copies of hot data can be kept in CXL memory to avoid memory thrashing.

Colloid, proposed by Vuppalapati et al. (SOSP 2024) adds another placement condition based on the following fact. Effective access latency of default tier might not be equal to hardware specified latency in some cases (multiple in-flight requests). Hence placement of hot data in default tier might not be as efficient as placing them in alternative tiers. Under some circumstances, the latency of default tier might be 2.5× more than alternative tiers. Colloid then balances page placements to improve average access latency.

Liu et al. (OSDI 2025) argue that hot data is not necessarily performance-critical. Out-of-order cores overlap independent misses, so accesses with high memory-level parallelism (MLP) hide most of the slow-tier latency, while low-MLP accesses, such as pointer chasing, pay all of it. The authors introduce the Amortized Offcore Latency, AOL = Latency/MLP, model the slowdown as $S = P \times K(\text{AOL})$, and build on it SOAR, an AOL-guided first-placement policy.

Our project aims to reproduce these claims by answering four questions:

- **Q1:** Can hotness be a faithful metric to measure performance?
- **Q2:** If not, how can a better metric be defined?
- **Q3:** With this metric, can we improve first data placement (SOAR)?
- **Q4:** If time permits, can these elements improve PolyRack?

The main focus of the project has not changed since the proposal. What has changed is our understanding of the platform: the server available to us is based on Intel Haswell, an older generation than the Skylake and Sapphire Rapids machines used by the authors, which affects how the metrics can be measured (Section 2.1).

2 Questions, Hypotheses, and Tests

2.1 Methodology and Testing Environment Updates

We launched basic commands on the remote server to fully characterize it. The results are reported below.

- CPU Intel Xeon E5-2680 v3. It has 2 physical processors, 12 cores per processor so 24 in total, 2 threads per core so 48 in total.
- There are 2 NUMA nodes, each having 126 GB of memory.
- The Kernel is an Ubuntu `6.8.0-147-generic` Kernel.
- Single memory tier (`memory_tier4`, nodes 0-1).
- Total memory of about 252 GB (2 × 126 GB)

The machines used for the experiments in the paper were:

- CloudLab dual-socket Intel Skylake server with 2 10-cores CPUs and 96 GB DD4 DRAM per socket. There, CXL is emulated by decreasing uncore frequency and disabling cores on one NUMA node.
- Intel Sapphire Rapids server with 32 cores and 192 GB DDR5 DRAM per socket, and ASIC-based 128 GB CXL memory expander.

In their experiment, they used 8 threads, with memory bandwidth 2.3–21 GB/s and RSS of 8–35 GB.

In the paper, they used the following counters to track AOL. LLC stands for Last Level Cache. They refer to LLC stalls as CPU stalls caused by LLC misses.

- Number of LLC stalls in L3 tier
- Number of cycles
- Cycles with pending requests
- Number of pending requests per cycle
- Number of requests to uncore

We now compare our machine with theirs. The CPUs are not the same, and this has two consequences. First, the machine has no CXL memory expander, and it does not expose an uncore-frequency driver, so we cannot slow down a node as the authors did on Skylake. Our slow tier is simply the remote NUMA node (node 1), which is only about 1.5× slower than local DRAM (Section 3.1), against about 2.1× for the paper's emulated tier. We therefore expect smaller slowdowns than in the paper. Second, our machine has 48 hardware threads; as in the paper, we limit the workloads to 8 threads with thread pinning.

We also noticed that we did not have all counters at hand. Hence, we need to make following modifications:

- The event `cycle_activity.stalls_l3_miss` does not exist on Haswell. We use `cycle_activity.stalls_l2_pending` as a proxy for the LLC stalls. It also counts stalls on L2 misses that hit in L3, so it overestimates P.
- The other counters are available.

2.2 Current Status of Research Questions

2.2.1 Q1: Can hotness be a faithful metric to measure performance?

Insight: out-of-order cores overlap independent misses, so high-MLP (often hot) accesses hide most of the slow-tier latency, while low-MLP accesses pay all of it.

Experiment: Run SoarAlto microbenchmark with sequential and pointer-chasing threads the hot/cold (migration disabled) in different configurations to prove that sequential on hot and pointer-chasing on cold is not optimal.

Status: Partial. We adapted the microbenchmark (thread pinning, explicit node allocation, per-thread timers) and measured the four placements (Section 3.2, Figure 1). With 10 runs under fixed CPU frequency and without NUMA balancing, the results support the hypothesis. The alternative H1′ (migration overhead) still needs a tiering system.

2.2.2 Q2: If not, how can a better metric be defined?

Insight: the average stall cycles per LLC miss vary by up to 4× across workloads: MLP amortizes the latency, so a predictor that ignores it (LLC stalls/cycle, P) overestimates the slowdown of high-MLP workloads.

Experiment: for each workload, measure the slowdown on the slow tier together with the counters for P, AOL and access counts; calibrate a, b of K with the sequential/pointer-chasing microbenchmarks; compare the Pearson coefficients of the three predictors.

Status: Partial. We measured P, AOL and the slowdown for the microbenchmark and for four GAPBS workloads (bc, bfs, pr, cc; slowdowns from +29% to +37%). The calibration of a, b and the comparison of the predictors have not started.

2.2.3 Q3: With this metric, can we improve first data placement (SOAR)?

Insight: if SOAR finds critical objects, it places them in DRAM at the beginning (first placement). Having optimal first allocations allows us to avoid unnecessary migrations during execution.

Experiment: compare the slowdown of SOAR vs. NoTier, TPP, NBT, Nomad and Colloid on the selected workloads, at 10–90% slow-tier ratios.

Status: Not started

2.2.4 Q4: Can these elements improve PolyRack?

Insight: PolyRack disaggregates memory into dedicated memory servers reached through CXL, which creates a fast/slow memory hierarchy similar to a tiered server.

Experiment: run the same workloads on PolyRack with and without SOAR, and compare against the traditional tiered server.

Status: Not started

2.3 Anomalies and Updates to the Hypothesis Tree

The hypothesis tree is unchanged, but we found three anomalies in the platform that affect how the hypotheses are tested:

- The LLC-stall event used by the paper does not exist on Haswell, so P is measured with a proxy (Section 2.1).
- The counter `offcore_requests_outstanding.demand_data_rd` returns inconsistent values on this machine, so we do not use it (Section 3.3).
- The emulated slow tier is only about 1.5× slower than local DRAM, so all slowdowns are smaller than in the paper.

A further hypothesis stated in the paper is the linear relationship between LLC stalls in fast tier and LLC stalls in slow tiers (part 3 of the article).

3 Progress, Results, and Analysis

3.1 Experimental Setup

We checked that we could access to the following counters of the machine:

- `cycle_activity.stalls_l2_pending` (LLC stalls, proxy)
- `cpu_clk_unhalted.thread` (number of cycles)

- `offcore_requests_outstanding.cycles_with_demand_data_rd` (cycles with pending requests)
- `offcore_requests_outstanding.demand_data_rd` (number of pending requests per cycle)
- `offcore_requests.demand_data_rd` (number of requests to uncore)

We also measured local and remote access latencies, as the time per step of the pointer-chasing loop (TLB misses included):

- local access latency: 102 ns
- remote access latency: 154 ns, i.e. about 1.5× the local one

3.2 E1: Hotness versus Criticality

We run a pointer-chasing thread (cold data) and a sequential read thread (hot data) at the same time, using the microbenchmark folder of the SoarAlto repository. Both threads run on node 0 (fast tier); the data of each thread is allocated on node 0 or node 1 (slow tier), which gives four configurations:

- All-on-DRAM: both on fast tier (baseline)
- All-on-Slow: both on slow tier
- Hot-on-DRAM: sequential (hot) on fast tier, pointer-chasing (cold) on slow tier
- Cold-on-DRAM: pointer-chasing (cold) on fast tier, sequential (hot) on slow tier

We want to show that cold data in fast tier and hot data in slow tier is more efficient than cold data in slow tier and hot data in fast tier, using the microbenchmark.

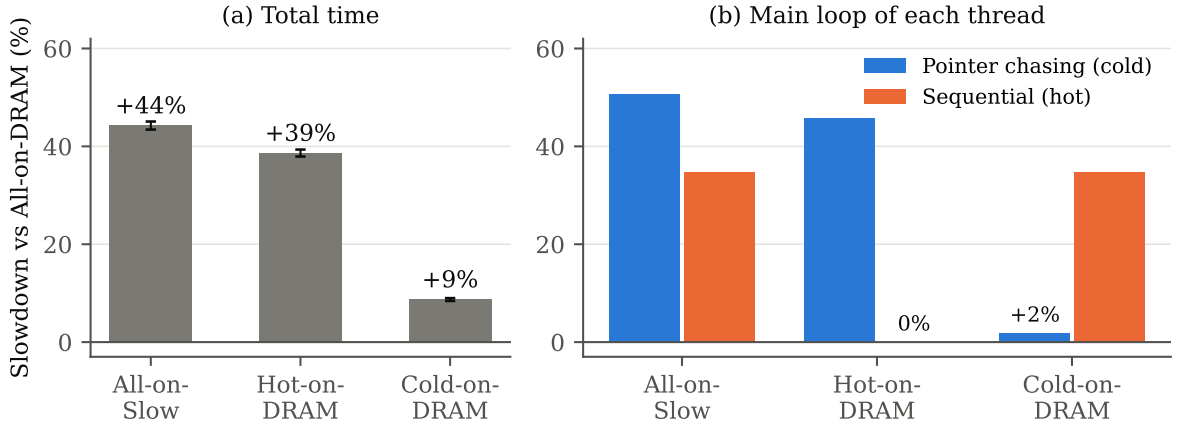


Figure 1: E1: slowdown of each placement with respect to All-on-DRAM. The pointer-chasing thread (1 GiB list, cold) and the sequential thread (1 GiB array, hot) run at the same time on two cores of node 0; node 1 is the slow tier. Governor `performance`, turbo off (2.5 GHz), NUMA balancing off. (a) Total program time, mean of 10 runs, error bars show the standard deviation. (b) Time of the main loop of each thread, excluding setup.

We ran 10 repetitions of the four configurations, interleaved, with the CPU governor set to `performance`, turbo disabled and automatic NUMA balancing disabled, and restored the

original settings afterwards. The measured frequency of the two cores was 2494 MHz in every configuration, and only 3 pages were migrated during the whole experiment.

All-on-DRAM takes 30.33 ± 0.34 s. Placing the cold pointer-chasing list on the fast tier (Cold-on-DRAM, +8.7%) is faster than placing the hot sequential array on the fast tier (Hot-on-DRAM, +38.6%): Hot-on-DRAM is 27.5% slower than Cold-on-DRAM (42.05 ± 0.22 s against 32.97 ± 0.09 s). All-on-Slow gives +44.3%.

The main loops alone (panel b) do not separate the two mixed configurations: the pointer-chasing loop loses 7.3 s when its list is on the slow tier, and the sequential loop loses 7.9 s when its array is. The difference in total time comes from the setup phase of the list (construction and shuffle), which is also latency-bound, and from the pointer-chasing thread being the one that determines when the program ends. Moreover, the sequential loop slows down by 35% on the slow tier, more than expected for a high-MLP pattern. Its throughput drops from about 11.0 to 8.2 GB/s, which suggests a bandwidth limit of the inter-socket link rather than a latency effect (to be verified).

These results support Hypothesis 1 on our setup. A first run of 3 repetitions with the default settings (turbo on) gave the same conclusion with slightly larger slowdowns. The alternative hypothesis H1′ (loss due to migration overhead) is not tested yet, since it requires a tiering system.

3.3 E2: Slowdown Predictors (Preliminary)

Our goal is to reproduce the following results. We must measure slowdown vs all on DRAM of our workloads, and find the right endogenous quantity to explain this slowdown, ie a quantity with the highest Pearson correlation coefficient. We would first measure $\frac{\text{LLC_stalls}}{\text{cycle}}$, compute the Pearson correlation coefficient, then do the same for $K(\text{AOL})\frac{\text{LLC_stalls}}{\text{cycle}}$. K links the extra LLC stalls on the slow tier to the LLC stalls on the fast tier (the paper argues that the LLC stalls on the slow tier are proportional to the LLC stalls on the fast tier). K is not a constant but a function of AOL, $K = 1/(a + b/\text{AOL})$, where a and b are calibrated with microbenchmarks (Figure 2).

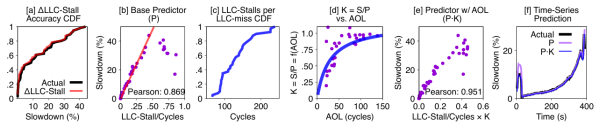


Figure 2: AOL tuning, reproduced from Liu et al. (OSDI 2025).

We derive the following metrics from the counters, with:

- O = `offcore_requests_outstanding.demand_data_rd`
- C = `offcore_requests_outstanding.cycles_with_demand_data_rd`
- N = `offcore_requests.demand_data_rd`
- Average latency (cycles per request): Latency = $\frac{O}{N}$
- Average MLP: MLP = $\frac{O}{C}$
- AOL = $\frac{\text{Latency}}{\text{MLP}} = \frac{C}{N}$
- Stall ratio $P = \frac{\text{cycle_activity.stalls_l2_pending}}{\text{cycles}}$

The counter O returns inconsistent values on this machine. Since O cancels out in AOL, we compute AOL directly as C/N, but we cannot measure the latency and the MLP separately.

The first measurements cover the microbenchmark and four GAPBS workloads (bc, bfs, pr, cc). Their slowdowns range only from +29% to +37%, which is too narrow to compare predictors, so we will add workloads with more varied MLP before computing the correlations.

3.4 SOAR: Feasibility

SOAR's profiling should run on the stock kernel: it relies on PEBS memory sampling, and we checked that `perf record -d` works on the machine. The rest of the SOAR pipeline has not been run yet. The baselines of E3 are different: TPP, NBT, Nomad and Colloid each require a specific kernel, so the kernel will have to be changed for each strategy, which needs the approval of the teaching team. The measurements will be made one strategy after the other, and repeated 10 times to make a statistical analysis.

4 Challenges

4.1 Hardware Differences

The CPUs and counters are not all the same. The LLC-stall event does not exist on Haswell, so we use `cycle_activity.stalls_l2_pending`, which overestimates P. The outstanding-requests counter is unreliable, so we only use AOL $= C/N$.

4.2 Tiered Memory Emulation and Baselines

There is no CXL memory implemented in the server. Hence, we emulate it with the second NUMA node: threads run on node 0 and the slow-tier data is allocated on node 1. This tier is only about 1.5× slower than local DRAM. Making it slower would require lowering the uncore frequency of node 1, which needs changes on the machine that we have not made yet. The baselines also require custom kernels (Section 3.4).

4.3 Authors' Artifact

The authors' benchmark code builds and runs on our machine, but not without changes. We modified the microbenchmark to pin the threads, allocate the data of each thread on a chosen node, and time each thread separately.

4.4 Help Needed from the Teaching Team

- Can we lower the uncore frequency of node 1 (through the `msr` module) to make the slow tier slower?
- Can we install custom kernels on the machine for the E3 baselines?
- Is there a way to get access to existing hardware of PolyRack, or emulate it? For emulation, we thought about simulating slowdowns depending on CPU placements on the rack.

5 Plan

We are a little bit late on the milestones: E1 is done except for the test of H1′, and the data collection for E2 has started. The main risks are the emulation of the slow tier and the kernels for the baselines, which depend on the answers of the teaching team (Section 4.4).

[TODO: Division of the work between the team members.]

Date	Milestone	Status
Nov. 12	Finish E1	Done (H1′ pending)
Nov. 19	Finish E2	Data collection started
Nov. 27	E3 on selected workloads and ratios	Not started
Dec. 4	If time permits E4 on PolyRack	Not started
Dec. 10	Poster deadline	
Dec. 17	Poster session	
Dec. 20	Final report deadline	

Table 1: Updated milestones.